

Alunos: Beatriz Christine Azevedo Batista; Francisco Felipe Barros Luz Santos; Giovani Artil Oliveira de Carvalho; Jorge Samuel Silva Coelho; Matheus Carvalho Reges; Renata Modesto Fernandes; Sofia Pinho Icaivino Moura; Vitória Luz Edwards

RELATÓRIO

MUNDO DOS BLOCOS VARIÁVEIS EM MODEL CHECKING

Parte I: Problema dos Blocos com Tamanho Variáveis.

Esta parte do relatório detalha a **extensão do modelo de planejamento STRIPS clássico** para um mundo de blocos que incorpora **tamanhos heterogêneos** e **restrições espaciais**, utilizando uma abordagem de Programação Lógica.

1. Limitações do Modelo Clássico STRIPS

O modelo clássico de mundo dos blocos utiliza uma representação abstrata baseada em relações lógicas, tipicamente `on(Block, Object)` e `clear(Object)`. Este modelo é insuficiente para problemas com restrições físicas realistas devido a incompatibilidades fundamentais:

- **Inadequação Espacial:** O novo domínio utiliza uma grade numerada (coordenadas 0 a 6), exigindo que a posição do bloco seja **absoluta**. O predicado `on/2` clássico é inadequado porque trata localizações como identificadores abstratos e **não consegue representar blocos de larguras variadas** que ocupam múltiplos espaços de coordenadas contíguos.
- **Ausência de Propriedades Físicas:** O modelo clássico assume que todos os blocos são uniformes. Ele carece de mecanismos para armazenar o **tamanho** dos objetos, o que significa que o operador `can/2` não pode impor restrições físicas cruciais, como a **estabilidade**. Isso permitiria planos que colocam um bloco largo sobre um estreito, o que é fisicamente ilegal.

2. Novo Esquema de Representação do Conhecimento

Para superar essas limitações, foi desenvolvido um novo esquema que modela o ambiente e os objetos de forma mais fundamentada:

Conceito	Modelo Clássico	Modelo Estendido	Detalhes Cruciais
Propriedades Estáticas	<code>bloco(X).</code>	<code>tamanho(X, L).</code>	Define a largura imutável do bloco (e.g., <code>tamanho(c, 2)</code>), fundamental para o raciocínio de estabilidade.
Ambiente	<code>lugar(N).</code> (Abstrato)	<code>table_slot(N).</code>	Define a mesa como uma grade linear e discretizada de slots endereçáveis (0 a 6).
Estado Dinâmico	<code>on(B, L)</code> ou <code>on(B, OB)</code>	<code>pos(Block, Support).</code>	Unifica a representação do suporte. O suporte pode ser: <code>table(X)</code> (captura a coordenada horizontal absoluta) ou <code>on(OtherBlock)</code> (captura o suporte vertical).
Raciocínio Espacial	Não existente	Conhecimento Derivado	Para manter o estado enxuto e não redundante, predicados auxiliares (<code>absolute_pos/3</code> , <code>is_free/2</code>) são usados para calcular informações complexas sob demanda, como quais slots da mesa estão ocupados por um bloco (<code>busy_slots/3</code>).

3. Redefinição dos Operadores de Planejamento e Leis Físicas

O operador de movimento foi redefinido para uma sintaxe orientada a objetivos: `move(Block, Destination)`. A responsabilidade de garantir a validade da ação é transferida inteiramente para o predicado `can/2`.

As pré-condições em `can/2` impõem rigorosamente as novas leis físicas:

1. **Acessibilidade do Bloco:** O bloco a ser movido deve estar `clear(Block)`.
2. **Verificação de Destino (em outro bloco - `on(TargetBlock)`):**
 - O `TargetBlock` deve estar `clear`.
 - **Estabilidade (Restrição Física Crucial):** A largura do bloco que está sendo movido (W_1) deve ser menor ou igual à largura do bloco de destino (W_2) ($\text{W}_1 \leq \text{W}_2$). Essa verificação podará qualquer tentativa de criar estruturas instáveis.
3. **Verificação de Destino (na mesa - `table(X)`):**
 - A coordenada X deve ser um slot válido.
 - **Disponibilidade de Espaço Contíguo:** O planejador deve usar o `size(Block, W)` e o predicado `is_free/2` para garantir que o intervalo completo de slots que o bloco ocuparia (de X a $X+W-1$) esteja totalmente desocupado antes de colocar o bloco.

As listas de atualização de estado (`adds/2` e `deletes/2`) são construídas dinamicamente para garantir que o fato `pos/2` seja atualizado e que o status `clear/1` do suporte anterior (se for um bloco) seja adicionado corretamente.

4. Impacto no Desempenho (Eficiência)

Embora o novo modelo exija mais trabalho computacional para a expansão de cada nó, a introdução de fortes restrições físicas no `can/2` atua como uma heurística poderosa.

Essas restrições podam o espaço de busca agressivamente. O planejador clássico poderia explorar longos ramos de planos sintaticamente válidos, mas fisicamente impossíveis (por exemplo, planos que começam com o empilhamento instável). O modelo estendido elimina esses caminhos inválidos imediatamente na raiz, assim que o `size_check` falha. O custo de verificar a restrição é trivial em comparação com o custo de explorar toda a subárvore inválida. Portanto, a incorporação dessas leis físicas leva a uma maior eficiência geral do planejamento.

O modelo fornece uma base robusta para extensões futuras, como planejamento em espaço tridimensional, uso de Programação Lógica de Restrições (CLP) para variáveis não instanciadas (como `table(X)`), e Planejamento de Ordem Parcial.

5. Tabelas Comparativas:

5.1. Tabela de Conceitos e Representação:

Conceito	STRIPS	Prolog Estendido	Proposta do modelo NUSMV	Justificativa para projeto NUSMV
Propriedades do Bloco	<code>block(X)</code>	<code>size(X,W).</code>	DEFINE <code>size_a:=1;</code>	Codifica dimensões físicas imutáveis como constantes de tempo de compilação, não variáveis de estado, para eficiência.
Mesa	<code>lugar(N)</code>	<code>slot_de_tabela(N)</code>	DEFINE <code>TABLE_SLOTS:=0..6;</code>	Passar de identificadores abstratos para uma grade espacial concreta.

Posição do Bloco	on(Bloco, Local)/ on (Bloco, OutroBloco)	pos(Bloco, tabela (x))/pos(Bloco, on(Other Block))	Var pos_Block: {table, on, block_id}; VAR x_Block: 0..6;	Para unificar a representação e capturar a posição horizontal absoluta.
Espaço Livre	limpar(Objeto)	clear_Block/ is_free(slot, state)	Var clear_Block: boolean;	Para distinguir entre folga vertical e folga horizontal.

5.2. Tabela de Restrições do Modelo Strips Clássico:

Restrição	Destino	Condição	Explicação	Suporte no Modelo
Mobility	N/D	Bloco deve estar limpo	Clear(Block) é membro do estado atual	move(Block, From, to)
Target Accessibility	\$TO\$(outro bloco ou lugar)	Destino deve estar limpo	Clear(to) é membro do estado atual	move(Block, From, to)
Position Confirmation	\$From\$	O bloco deve estar em De	on(Block, From) é membro do estado atual	move(Block, From, to)
Stability	N/D	Não há verificação de estabilidade física.	Não há propriedades físicas como tamanho físicas como tamanho.	Omissão de can/2

5.3: Tabela de Restrições do Modelo Estendido:

Restrição	Destino	Regra em Ling Natural	Implem. NuSMV
Mobility	N/D	O bloco a ser movido não pode ter nada em cima dele	State.clear [Block] = TRUE
Target Accessibility	\$on(BlocoAlvo)\$	O bloco alvo deve estar limpo para receber outro	State.clear [TargetBlock] = TRUE
Stability	\$on(BlocoAlvo)\$	Um bloco só pode ser colocado sobre outro bloco de largura igual ou maior	size[Block]<= size[TargetBlock]
Spatial Occupancy	\$table(x)\$	Deve haver um bloco contíguo de slots livres na mesa.	space_check(Block, X) = TRUE
Logical Validity	\$on(BlocoAlvo)\$	Um bloco não pode ser colocado sobre si mesmo	Block \neg target Block

Parte II: PLANEJAMENTO (PLANNING) COM NUSMV

O tema principal aqui é uma técnica avançada que usa a Verificação de Modelo (Model Checking) para resolver problemas de Planejamento. A ferramenta que faz isso é chamada NuSMV.

1. Problema de Planejamento:

A ideia do planejamento é bem intuitiva, como se fosse um quebra-cabeça.

1. Estado Inicial (\$I\$): Você tem a situação de onde você começa (o ponto de partida).
2. Estado Meta (\$G\$): Você tem o objetivo que deseja alcançar (o ponto de chegada).
3. Operadores de Ação (\$OP\$): Você tem as regras ou movimentos que pode fazer (as peças do quebra-cabeça).

O problema, então, é encontrar uma sequência de ações (\$o_1, o_2, \dots, o_n\$) que te leve de forma válida do Estado Inicial (\$I\$) ao Estado Meta (\$G\$). Um exemplo é o contexto de mover blocos, onde definimos as precondições (o que precisa ser verdade para a ação acontecer) e os efeitos (o que muda depois da ação).

2. A Estratégia Genial: Usando Model Checking.

Em vez de procurar a solução diretamente, usa-se uma "estratégia reversa". Codifica-se o problema de planejamento (como as regras do jogo e os estados) no NuSMV.

2.1. A Especificação (O "Não Objetivo"):

Diz-se para o NuSMV que o que queremos verificar é se: "O objetivo NÃO é alcançável".

- Se o NuSMV provar que essa afirmação é VERDADEIRA, significa que não há solução, e não se chega ao seu objetivo.

- Mas, se o NuSMV provar que essa afirmação é FALSA (ou seja, o objetivo pode ser alcançado), ele automaticamente gera um contraexemplo.

2.2. O Contraexemplo Vira o Plano:

Esse contraexemplo que o NuSMV produz é, na verdade, a sequência exata de passos (o plano) para ir do início ao fim! É uma maneira muito eficiente de encontrar soluções.

2.3. Exemplos Clássicos Modelados:

Essa técnica é aplicada a três jogos famosos:

- **A Torre de Hanói (The Tower of Hanoi):**

Este é um jogo matemático com três pinos e N discos de tamanhos diferentes. O objetivo é mover toda a pilha, que começa no pino esquerdo, para o pino direito.

As Regras (o que o modelo precisa saber):

1. Só se pode mover um disco por vez.
2. O movimento só pode ser feito com o disco que está no topo.
3. A regra mais importante: Nunca se pode colocar um disco em cima de um disco menor.

O problema é codificado definindo a posição de cada disco (d_1, d_2, d_3, d_4) como variáveis. O estado inicial é quando todos os discos estão no pino esquerdo ($d_1 = \text{left} \ \& \ d_2 = \text{left} \ \& \ d_3 = \text{left} \ \& \ d_4 = \text{left}$). A especificação (para encontrar a solução) é: $\text{CTLSPEC} \ ! \ \text{EF} \ (d_1=\text{right} \ \& \ d_2=\text{right} \ \& \ d_3=\text{right} \ \& \ d_4=\text{right})$. Se essa negação for falsa, temos o plano de solução.

- **O Barqueiro (The Ferryman):**

Neste problema, um barqueiro precisa atravessar um rio levando um lobo, uma cabra e um repolho.

O Desafio de Segurança:

- Ele só pode levar no máximo um item por vez.

- Ele não pode deixar a cabra e o repolho sozinhos (a cabra come o repolho).
- Ele não pode deixar o lobo e a cabra sozinhos (o lobo come a cabra).

As variáveis cabbage, goat, wolf e man registram se eles estão na margem direita ou esquerda. O modelo define o que é um estado seguro (safe_state): se a cabra estiver com o lobo OU a cabra estiver com o repolho, a cabra deve estar com o barqueiro (goat = man). O objetivo é que todos estejam na margem esquerda. A especificação usada para buscar o plano seguro é: CTLSPEC ! E[safe_state U goal].

- **Jogo da Velha:**

O Jogo da Velha (ou Jogo do Galo) também pode ser modelado para analisar estratégias.

- O tabuleiro é um array de 9 posições, onde "0" é vazio, "1" é o Jogador 1 e "2" é o Jogador 2.
- Os jogadores se revezam (player : 1..2).
- Definimos quando o Jogador 1 ganha (win1) ou o Jogador 2 ganha (win2), e quando é empate (draw).

Podemos usar o NuSMV para verificar estratégias. Por exemplo, podemos testar se o Jogador 2 tem uma "estratégia não perdedora" (non-losing strategy), usando uma especificação CTL (a linguagem de lógica temporal) bem complexa: CTLSPEC AX (EX (AX (EX (AX (EX (AX (EX (AX !win1))))))). Isso permite analisar, por exemplo, se o Jogador 2 consegue evitar a derrota, mesmo que o Jogador 1 faça um movimento inicial específico (como preencher a posição 5).

Em resumo, a Verificação de Modelo com NuSMV não serve apenas para provar que algo está certo ou errado, mas também, de forma muito prática, para descobrir como fazer algo – gerando o plano (a solução) através de um contraexemplo.

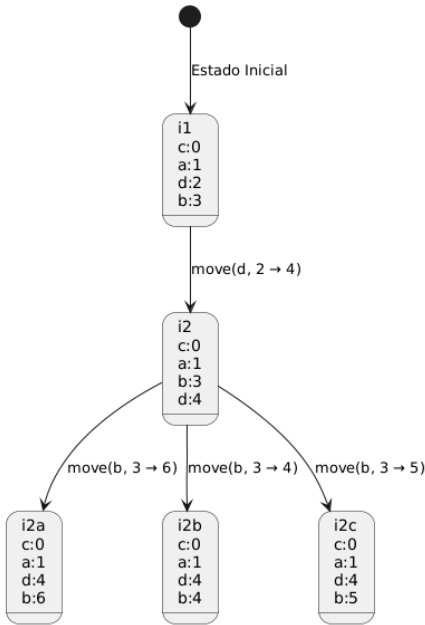
Parte III: Planejador para Empilhar Blocos de Diferentes Posições:

Este trabalho exige respostas a diversos pontos específicos referentes às figuras das Situações 1, 2 e 3:

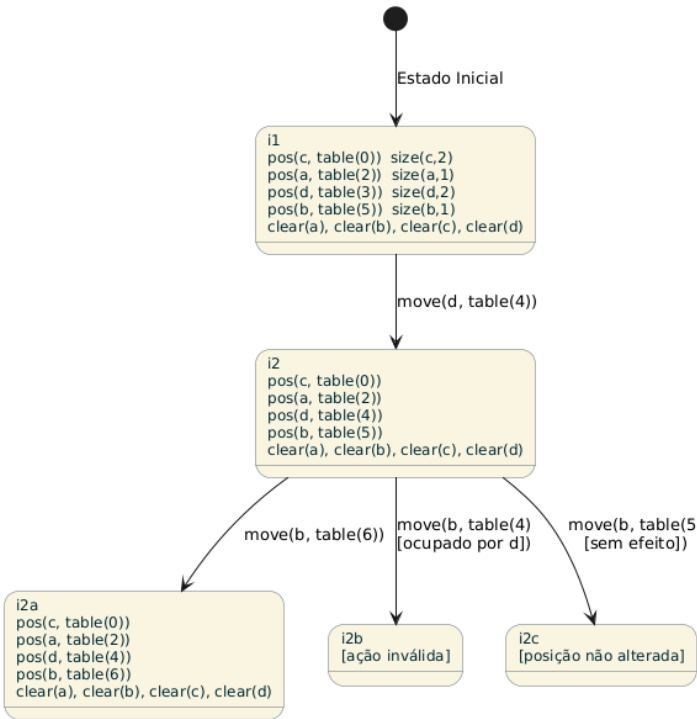
1. **Linguagem de Representação:** Propor uma linguagem para esta instância do mundo dos blocos, detalhando:
 - Tamanho do bloco.
 - Posição na mesa (com medida de espaço lateral e vertical).
 - Condições de **vacância** (para preenchimento do espaço, vertical e horizontal).
 - Condições de **estabilidade** (para um bloco sobre outro, incluindo vacância no topo e centro de massa em caso de bloco maior sobre menor).
 - A proposta deve ser justificada comparando seus elementos com a Figura 17.1 do Capítulo 17.
 - *Dica:* Representar os espaços em uma grade onde o menor bloco caiba em um espaço.
2. **Modificação do Planejador:** Modificar o código do planejador da Figura 17.6 para manipular corretamente variáveis sobre *goals* e ações, conforme a discussão na seção 17.5. Esta mudança deve ser indicada e explicada.
3. **Geração Manual de Plano (Situação 1):** Gerar manualmente, usando a linguagem proposta, o plano de ações para ir do estado inicial (*s_inicial*) ao estado final (*s_final*) em três cenários da Situação 1:
 - De s_1 até s_2 .
 - De s_2 até $s_2(a)$.
 - De s_2 até $s_2(b)$ (mencionado duas vezes, pontos 3. e 4. do trabalho).
4. **Análise das Situações 2 e 3:** Analisar o que é necessário para o planejador gerar planos de ações de s_0 até o estado final de cada situação. Essa análise deve considerar a linguagem de representação proposta e os padrões de situações dos blocos.

Os diagramas de estados gerados para este trabalho seguem a seguir:

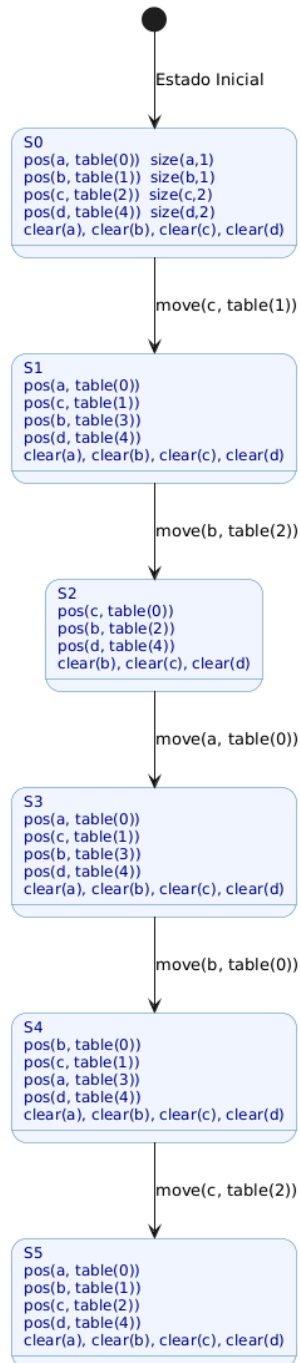
Diagrama de Estados - Situação 1



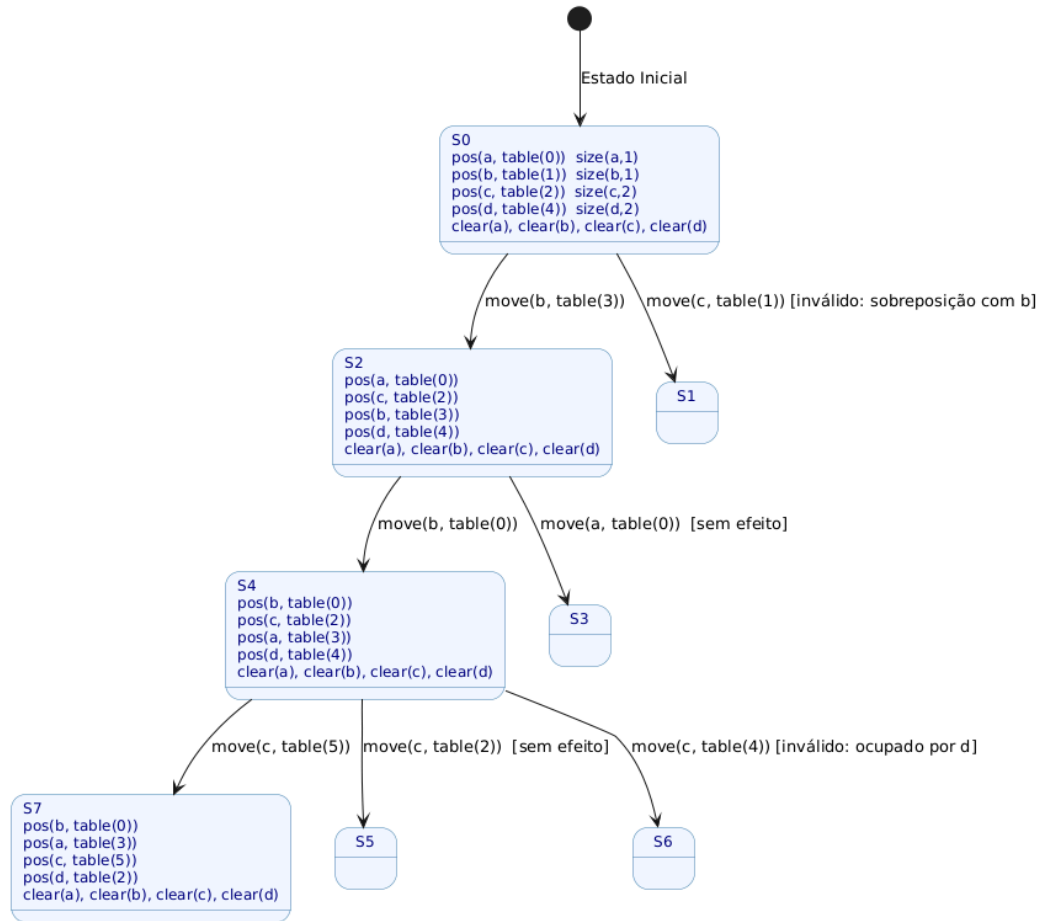
Planejamento - Situação 1 com Tamanhos Variáveis



Planejamento - Situação 2 com Tamanhos Variáveis



Planejamento - Situação 2 com Tamanhos Variáveis



Planejamento - Situação 3 com Tamanhos Variáveis

